Pandas I: loading and inspecting data

🎯 Today's goal

By the end of this hour you will:

- know the two pandas building blocks: the **Series** (one column) and the **DataFrame** (a table)
- **load** a CSV file into a DataFrame
- **inspect** it: its first rows, size, column types and quick statistics
- pick columns, rows and single values with `[ ]`, `.loc` and `.iloc`
- count values, add a calculated column, sort, and save the table again

🤔 Why this matters

pandas is the most used data tool in Python. If NumPy is a calculator for grids of numbers, pandas is a **spreadsheet you control with code**: its tables have column names, can mix numbers, text and dates, and come with hundreds of ready-made tools.

The first thing a data scientist does with any new dataset is exactly today's topic: load it and **look at it**. How many rows? What columns? Which types? Any strange values? You can't analyse data you haven't looked at.

🐼 Series and DataFrame

Everyone imports pandas with the nickname `pd`. A **Series** is one column of values with a **label** for each value. The labels are called the **index**. If you don't give any, pandas numbers the rows 0, 1, 2, ...

```
import pandas as pd

# Made-up afternoon temperatures, labelled by day
temps = pd.Series([21.5, 24.0, 19.5, 26.0], index=["Mon", "Tue", "Wed", "Thu"])

print(temps)
print("Tuesday:", temps["Tue"])
print("Average:", temps.mean())
```

```
Mon    21.5
Tue    24.0
Wed    19.5
Thu    26.0
dtype: float64
Tuesday: 24.0
Average: 22.75
```

A **DataFrame** is a table: several Series side by side, sharing the same index. The easiest way to make one is from a dictionary: each key becomes a column name, each list a column:

```
# Made-up data about three cities
cities = pd.DataFrame({
    "city": ["Cairo", "Lagos", "Lima"],
    "temperature": [31, 29, 19],
    "rainy": [False, True, False],
})

cities
```

	city	temperature	rainy
0	Cairo	31	False
1	Lagos	29	True
2	Lima	19	False

The bold numbers on the left (0, 1, 2) are the index: the row labels.

📂 Loading a CSV file

Real data usually arrives as a file. `pd.read_csv("file.csv")` reads a CSV file into a DataFrame in one line. This page can't reach files on your computer, so we write the CSV text right into the code

and use `io.StringIO`, which makes a piece of text **behave like a file**. On your own computer you'd just write `pd.read_csv("cafe_sales.csv")`.

```
import io

# Made-up sales of a small café: one row per order
csv_text = """order_id,day,drink,size,price,qty
1,Mon,coffee,large,3.50,2
2,Mon,tea,small,2.00,1
3,Mon,latte,medium,3.20,1
4,Mon,coffee,small,2.50,3
5,Tue,juice,medium,3.00,2
6,Tue,coffee,medium,3.00,1
7,Tue,tea,large,2.60,2
8,Tue,latte,large,3.80,1
9,Wed,coffee,large,3.50,4
10,Wed,coffee,small,2.50,1
11,Wed,juice,small,2.40,2
12,Wed,tea,medium,2.30,1
13,Thu,latte,medium,3.20,2
14,Thu,coffee,medium,3.00,2
15,Thu,tea,small,2.00,3
16,Thu,juice,large,3.60,1
17,Fri,coffee,large,3.50,3
18,Fri,latte,small,2.70,2
19,Fri,coffee,medium,3.00,2
20,Fri,tea,large,2.60,1
21,Sat,juice,medium,3.00,4
22,Sat,coffee,small,2.50,2
23,Sat,latte,large,3.80,3
24,Sat,coffee,large,3.50,1
"""

sales = pd.read_csv(io.StringIO(csv_text))
sales.head()      # the first 5 rows
```

	order_id	day	drink	size	price	qty
0	1	Mon	coffee	large	3.5	2
1	2	Mon	tea	small	2.0	1
2	3	Mon	latte	medium	3.2	1
3	4	Mon	coffee	small	2.5	3
4	5	Tue	juice	medium	3.0	2

Three quote marks `"""` start and end a string that can span many lines.

🔍 First look at the data

`head()` shows the first 5 rows, `tail()` the last ones (give a number for more or fewer). Then three questions: how big is it, what are the columns, and what type is each column?

```
print(sales.tail(3))
print()
print("shape:", sales.shape)           # (rows, columns), like NumPy
print("columns:", list(sales.columns))
print()
print(sales.dtypes)
```

```
    order_id  day   drink   size  price  qty
21        22  Sat  coffee  small    2.5    2
22        23  Sat   latte  large    3.8    3
23        24  Sat  coffee  large    3.5    1

shape: (24, 6)
columns: ['order_id', 'day', 'drink', 'size', 'price', 'qty']

order_id      int64
day             str
drink           str
size            str
price       float64
qty           int64
dtype: object
```

pandas guessed the types while reading: `int64` for whole numbers, `float64` for decimals, and `str` for text. If a column you expected to be numbers shows up as `str`, some value in it isn't a clean number (Day 12 is all about fixing that).

`info()` gives the same overview in one go, plus the number of **non-null** (not missing) values in each column, so you can spot gaps:

```
sales.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 24 entries, 0 to 23
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   order_id  24 non-null     int64
 1   day       24 non-null     str
 2   drink     24 non-null     str
 3   size      24 non-null     str
 4   price     24 non-null     float64
 5   qty       24 non-null     int64
dtypes: float64(1), int64(2), str(3)
memory usage: 936.0 bytes
```

`describe()` calculates quick statistics for every number column: the count, mean, standard deviation (`std`), minimum, maximum and the **quartiles** (`25%`, `50%`, `75%`: a quarter, half and three quarters of the values are below these numbers; `50%` is the median from Day 7):

```
sales.describe()
```

	order_id	price	qty
count	24.000000	24.000000	24.000000
mean	12.500000	2.945833	1.958333
std	7.071068	0.538096	0.954585
min	1.000000	2.000000	1.000000
25%	6.750000	2.500000	1.000000
50%	12.500000	3.000000	2.000000
75%	18.250000	3.500000	2.250000
max	24.000000	3.800000	4.000000

Already you can read a lot: prices go from 2.00 to 3.80, and most orders are for 1 to 3 drinks. (The statistics for `order_id` mean nothing: it's just a counter. Not every number column is a measurement.)

🎯 Picking columns

One column name in square brackets gives a **Series**. A **list** of column names (notice the double brackets) gives a smaller **DataFrame**:

```
drinks = sales["drink"]              # one column: a Series
print(type(drinks))
print(drinks.head(3))

sales[["drink", "price", "qty"]].head(3)    # several columns: a DataFrame
```

```
<class 'pandas.Series'>
0    coffee
1       tea
2     latte
Name: drink, dtype: str
```

	drink	price	qty
0	coffee	3.5	2
1	tea	2.0	1
2	latte	3.2	1

Column names must be spelled exactly right, capital letters included. A mistake on purpose:

💡 This example has a mistake on purpose. Run it and read the error.

```
sales["Drink"]
```

⚠️ **KeyError**

That key or column name doesn't exist. Check the spelling.

```
Traceback (most recent call last):
  line 1, in <module>
    sales["Drink"]
    ~~~~~^^^^^^^^^
KeyError: 'Drink'
```

Read the **last line**: `KeyError: 'Drink'` means "no column with this name". Print `sales.columns` and compare.

📍 Picking rows and cells: `.loc` and `.iloc`

`.loc[rows, columns]` picks by **label** (index labels and column names). `.iloc[rows, columns]` picks by **position** (0, 1, 2, ..., like a list). Here the row labels happen to be 0, 1, 2..., so they look alike, but on Day 11 you'll see when they differ:

```
print(sales.loc[0, "drink"])     # one cell: row label 0, column "drink"
print(sales.iloc[-1, 2])         # one cell: last row, column at position 2
print()
print(sales.loc[0:2, ["drink", "qty"]])   # rows 0 to 2: 2 INCLUDED
print()
print(sales.iloc[0:2, 0:3])      # rows 0-1, columns 0-2: end NOT included
```

```
coffee
coffee

    drink  qty
0  coffee    2
1     tea    1
2   latte    1

   order_id  day   drink
0         1  Mon  coffee
1         2  Mon     tea
```

Watch out: a slice with `.loc` *includes* the end label (`0:2` gives 3 rows), while `.iloc` works like a normal Python slice and *excludes* the end (`0:2` gives 2 rows).

🔢 Counting values

For text columns, the most useful question is "how often does each value appear?". `value_counts()` answers it, sorted from most to least common. `unique()` lists the different values and `nunique()` counts them:

```
print(sales["drink"].value_counts())
print()
print("Different sizes:", list(sales["size"].unique()))
print("How many different days:", sales["day"].nunique())
```

```
drink
coffee    10
tea        5
latte      5
juice      4
Name: count, dtype: int64

Different sizes: ['large', 'small', 'medium']
How many different days: 6
```

`value_counts()` returns a Series: the drinks are its index and the counts its values. So `sales["drink"].value_counts()["tea"]` gives the number of tea orders, and `.index[0]` gives the most common drink.

➕ Adding a column

Columns work like NumPy arrays: maths happens on every row at once. Assigning to a new column name adds the column:

```
sales["total"] = sales["price"] * sales["qty"]

# Changing ONE cell: use .loc[row, column]
sales.loc[0, "qty"] = 2      # (it was already 2; just showing how)

sales.head()
```

	order_id	day	drink	size	price	qty	total
0	1	Mon	coffee	large	3.5	2	7.0
1	2	Mon	tea	small	2.0	1	2.0
2	3	Mon	latte	medium	3.2	1	3.2
3	4	Mon	coffee	small	2.5	3	7.5
4	5	Tue	juice	medium	3.0	2	6.0

pandas 3 rule: to change a value, always use `.loc[row, column] = value`. The look-alike `sales["qty"][0] = 2` (two pairs of brackets one after the other) changes a temporary copy and leaves your table **unchanged**. This behaviour is called *copy-on-write*.

`sort_values()` sorts the rows by a column. `ascending=False` puts the biggest first (you'll do much more sorting tomorrow):

```
sales.sort_values("total", ascending=False).head(5)
```

	order_id	day	drink	size	price	qty	total
8	9	Wed	coffee	large	3.5	4	14.0
20	21	Sat	juice	medium	3.0	4	12.0
22	23	Sat	latte	large	3.8	3	11.4
16	17	Fri	coffee	large	3.5	3	10.5
3	4	Mon	coffee	small	2.5	3	7.5

💾 Saving to CSV

`to_csv()` writes the table to a file. `index=False` leaves out the row numbers, which you rarely want in the file. As on Day 7, files on this page live in the page's memory only:

```
sales.to_csv("cafe_sales_with_totals.csv", index=False)

again = pd.read_csv("cafe_sales_with_totals.csv")
print("Loaded back:", again.shape)
print(list(again.columns))
```

```
Loaded back: (24, 7)
['order_id', 'day', 'drink', 'size', 'price', 'qty', 'total']
```

Compare that with Day 7, where saving and loading a table took a `csv.writer`, a loop and `float()` conversions. pandas does it all, and even guesses the types for you.

⚠️ Common mistakes

- Misspelling a column name (capitals matter) → `KeyError`. Check `df.columns`.

- Single vs double brackets: `df["a"]` is a Series, `df[["a", "b"]]` is a DataFrame.
- Forgetting that `.loc` slices include the end, while `.iloc` slices don't.
- Changing a value with `df["col"][0] = …`: in pandas 3 this does nothing. Use `df.loc[0, "col"] = …`.
- Writing `df.shape()` with brackets: `shape` and `columns` are not functions, `head()` and `describe()` are.

✅ Recap

- `import pandas as pd`; a **Series** is one labelled column, a **DataFrame** is a table.
- `pd.DataFrame({...})` from a dict; `pd.read_csv(...)` from a file (here via `io.StringIO`).
- Inspect with `head()`, `tail()`, `shape`, `columns`, `dtypes`, `info()`, `describe()`.
- `df["col"]`, `df[["a", "b"]]`, `df.loc[label, "col"]`, `df.iloc[row, col]`.
- `value_counts()`, `unique()`, `nunique()` for counting.
- `df["new"] = df["a"] * df["b"]` adds a column; `sort_values("col", ascending=False)` sorts; `to_csv(name, index=False)` saves.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. How big is it?

Load the made-up library data into a DataFrame called `books`. Then store its number of rows in `n_rows` and its number of columns in `n_cols`, using `.shape`.

```
import io
import pandas as pd

csv_text = """title,genre,pages,loans
River Song,novel,320,14
Stars Above,science,210,9
Quiet Lake,novel,280,22
Bread Basics,cooking,150,5
Code for All,science,400,17
Old Roads,history,350,3
"""

# Load books, then create n_rows and n_cols
```

Hints:

1. `books = pd.read_csv(io.StringIO(csv_text))`
2. `books.shape` is a pair `(rows, columns)`. Take `books.shape[0]` and `books.shape[1]`.

2. Pick the columns

From the made-up `students` table, create:

- `scores`: the `score` column only, as a **Series**
- `small`: a **DataFrame** with only the columns `name` and `score`, in that order
- `second_name`: the name in the row with label `1`, using `.loc`

```
import pandas as pd

students = pd.DataFrame({
    "name": ["Amira", "Ben", "Chen", "Dana"],
    "age": [21, 23, 22, 20],
    "city": ["Cairo", "Lagos", "Lima", "Oslo"],
    "score": [82, 67, 91, 74],
})

# Create scores, small and second_name
```

Hints:

1. One column: single brackets, `students["score"]`.
2. Several columns: a list inside the brackets, `students[["name", "score"]]`.
3. `second_name = students.loc[1, "name"]`

3. Add a total column

Add a column called `total` to the made-up `orders` table, holding `price` × `qty` for every row. Then store the **sum** of the whole new column in a variable called `revenue`.

```
import pandas as pd

orders = pd.DataFrame({
    "item": ["pen", "notebook", "bag", "ruler", "glue"],
    "price": [1.20, 3.50, 24.00, 0.80, 2.10],
    "qty": [10, 4, 1, 6, 3],
})

# Add the total column, then create revenue
```

Hints:

1. `orders["total"] = orders["price"] * orders["qty"]`
2. A column has `.sum()`, just like a NumPy array: `orders["total"].sum()`

4. Count the cities

The made-up table lists where online customers live. Create:

- `city_counts` : how many customers each city has (use `value_counts()`)
- `top_city` : the name of the most common city
- `n_cities` : how many **different** cities there are

```
import pandas as pd

customers = pd.DataFrame({
    "customer": [1, 2, 3, 4, 5, 6, 7, 8, 9],
    "city": ["Lima", "Cairo", "Lima", "Oslo", "Lima",
             "Cairo", "Pune", "Oslo", "Lima"],
})

# Create city_counts, top_city and n_cities
```

Hints:

1. `city_counts = customers["city"].value_counts()`
2. `value_counts()` is sorted from most common, so the top city is `city_counts.index[0]` .
3. `n_cities = customers["city"].nunique()`

5. Challenge: clinic summary

Load the made-up clinic visits into a DataFrame called `visits`, then create:

- `mean_wait`: the average `wait_minutes`, rounded to 1 decimal
- `max_wait`: the longest wait
- `top_department`: the department with the most visits
- `longest_patient`: the `patient` who waited the longest (hint: sort!)

```
import io
import pandas as pd

csv_text = """patient,department,wait_minutes
P01,eyes,25
P02,skin,40
P03,eyes,15
P04,heart,55
P05,eyes,30
P06,skin,20
P07,heart,70
P08,eyes,10
"""

# Load visits, then create mean_wait, max_wait,
# top_department and longest_patient
```

Hints:

1. `visits["wait_minutes"].mean()` and `.max()`; round with `round(..., 1)`.
2. The most common department: `visits["department"].value_counts().index[0]`.
3. Sort with the longest wait first, then take the first row's patient: `visits.sort_values("wait_minutes", ascending=False).iloc[0]["patient"]`.

➡️ Tomorrow

Pandas II: filtering, sorting and grouping. Today you looked at the whole table. Tomorrow you'll ask it questions: "only the orders above 10 dollars", "the top 3 products", "total sales per store".

You'll filter rows with conditions, sort by several columns and summarise groups with `groupby`.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. How big is it?

```
import io
import pandas as pd

csv_text = """title,genre,pages,loans
River Song,novel,320,14
Stars Above,science,210,9
Quiet Lake,novel,280,22
Bread Basics,cooking,150,5
Code for All,science,400,17
Old Roads,history,350,3
"""

books = pd.read_csv(io.StringIO(csv_text))
n_rows = books.shape[0]
n_cols = books.shape[1]
```

2. Pick the columns

```
import pandas as pd

students = pd.DataFrame({
    "name": ["Amira", "Ben", "Chen", "Dana"],
    "age": [21, 23, 22, 20],
    "city": ["Cairo", "Lagos", "Lima", "Oslo"],
    "score": [82, 67, 91, 74],
})

scores = students["score"]
small = students[["name", "score"]]
second_name = students.loc[1, "name"]
```

3. Add a total column

```
import pandas as pd

orders = pd.DataFrame({
    "item": ["pen", "notebook", "bag", "ruler", "glue"],
    "price": [1.20, 3.50, 24.00, 0.80, 2.10],
    "qty": [10, 4, 1, 6, 3],
})

orders["total"] = orders["price"] * orders["qty"]
revenue = orders["total"].sum()
```

4. Count the cities

```
import pandas as pd

customers = pd.DataFrame({
    "customer": [1, 2, 3, 4, 5, 6, 7, 8, 9],
    "city": ["Lima", "Cairo", "Lima", "Oslo", "Lima",
             "Cairo", "Pune", "Oslo", "Lima"],
})

city_counts = customers["city"].value_counts()
top_city = city_counts.index[0]
n_cities = customers["city"].nunique()
```

5. Challenge: clinic summary

```
import io
import pandas as pd

csv_text = """patient,department,wait_minutes
P01,eyes,25
P02,skin,40
P03,eyes,15
P04,heart,55
P05,eyes,30
P06,skin,20
P07,heart,70
P08,eyes,10
"""

visits = pd.read_csv(io.StringIO(csv_text))
mean_wait = round(visits["wait_minutes"].mean(), 1)
max_wait = visits["wait_minutes"].max()
top_department = visits["department"].value_counts().index[0]
longest_patient = visits.sort_values("wait_minutes", ascending=False).iloc[0]
["patient"]
```